EEE 475/575 Medical Image Reconstruction & Processing
Homework #4
29 April 2026, Wednesday at 23:59

GUIDELINES FOR HOMEWORK SUBMISSION

- Upload your solutions to Moodle as **two separate files**:
  1) **PDF Report File:** The report should be typeset (i.e., no handwriting) and should properly present your results. In addition, the report should display the relevant sections of your code at the end of each subpart. No partial credits will be given to answers with missing codes or unjustified answers.
     At the beginning of your report, provide a section titled "GenAI Usage Disclosure" and clearly explain whether you have used any genAI tools. Include the name/version of the tool and a brief description of how it was used (see the "GenAI Policy" included below).
  2) **Code File:** You can use any programming platform of your choice. If you do not upload your code file, your homework will not be evaluated (i.e., 0 pts).
- Submission system will remain open for 1 day after the deadline:
  - No points will be lost if you submit your assignment within 12 hours past the deadline.
  - There will be a 50% penalty if you submit after 12 hours but within 24 hours past the deadline.
  - No submissions beyond 24 hours past the deadline.

GenAI Policy: Generative AI (genAI) tools (e.g., large language models, coding assistants) may be used for homework assignments and project to assist with coding and to improve writing. Any use of genAI must be clearly disclosed, including the name/version of the tool and a brief description of how it was used. Upon request, the student should be ready to provide prompts, outputs, and execution logs pertaining to their genAI use for all assignments. Students remain fully responsible for all submitted material and must be able to explain and justify their code, results, and written content. Reliance on genAI does not transfer responsibility for correctness or originality. Failure to disclose AI usage or inappropriate reliance on AI-generated content will be treated as a violation of academic integrity.

PREPARATION

From the Moodle page of the class, download the following file:

- **multicoil-data.mat:** Contains fully-sampled MRI data acquired in a line-by-line fashion using an 8-channel head coil. im(Nx,Ny,Nc) contains the fully-sampled images from all coils, map1(Nx,Ny,Nc) contains the estimated coil sensitivities, map2(Nx,Ny,Nc) contains a second set of (slightly wrong) estimated coil sensitivities. Here, Nx = 224 and Ny = 160 are image dimensions, and Nc = 8 is the number of receive coils.

PART I – MULTI-COIL RECONSTRUCTION (20 pts)

In this homework, for simplicity, we will assume that the noise in each coil is independent and identically distributed with unit variance (i.e., $\Psi = I$), so that we can ignore the noise covariance matrix Ψ in our computations and derivations.

1.1) Display MRI Images: Display the magnitude and phase of the images from all coils. Display the corresponding k-space spectrums for all coils (see Homework #1 for details on displaying the spectrum). Display the magnitude and phase of the coil sensitivities for all coils, for both **map1** and **map2**. Briefly comment on the differences between the coil sensitivities from map1 and map2.

Note: Throughout this homework, use the "*montage*" function (see Homework #3 for details) whenever you need to display images from all coils in a compact form. Adjust the display windowing appropriately to see the brain tissue (see *'DisplayRange'* parameter of *montage*).

1.2) Multi-Coil Reconstruction: Implement optimally weighted linear combination (OLC) across coils using **map1**. OLC is expressed as followes (assuming $\Psi = I$):

$$\hat{m} = (C^H C)^{-1} C^H \underline{m_s} = \frac{1}{\sum_l |c_l|^2} \sum_l c_l{}^* m_l$$

For a given voxel, $\hat{m}$ is the OLC voxel intensity, C is the vector of coil sensitivities, and $\underline{m_s}$ is the vector of voxel intensities for images from all coils, c_l is coil sensitivity for l-th coil, m_l is the voxel intensity for the image from l-th coil. Here, the weight for each coil is equal to the ratio of the complex conjugate of the respective coil sensitivity to the sum-of-squares combination of the sensitivities across all coil.

Display the magnitude image for OLC using map1.

Note #1: The coil sensitivities are zero in the background regions, which may yield NaN pixel values during reconstructions. Set all NaN pixels to zero as the last step of the reconstructions. Do this for all reconstructions that yield NaN values in this homework.

Note #2: PSNR and SSIM are very sensitive to normalization, especially if the compared images come from different reconstruction methods or from different coil sensitivities. Firstly, normalize the reconstructed image by its 95th percentile pixel intensity (as this value is less sensitive to reconstruction errors than the maximum value). You can use "*prctile"* built-in function of MATLAB to compute the 95th percentile pixel intensity for a given image *im* as follows: prctile(im(:),95).

Note #3: We will use OLC from **map1** (after 95th percentile normalization) as the *reference image* in Part I and Part II. Define a variable named max_val as the maximum pixel intensity of this image. During PSNR and SSIM computations, normalize both the image of interest and the reference image by max_val.

1.3) Perform OLC using map2. Perform 95th percentile normalization. Display the magnitude image using *imshow* grayscale window of [0 max_val] to enable visual comparison. Display the error image with respect to the reference image using *imshow* grayscale window of [0 0.2*max_val]. Compute PSNR and SSIM. Briefly comment on the results.

1.4) SoS Multi-coil Reconstruction: Implement square root of the sum-of-squares (SoS) combination across coils (see the lecture notes):

$$\hat{m}_{SoS} = \sqrt{\underline{m}_s{}^H \, \underline{m}_s} = \sqrt{\sum_l |m_l|^2}$$

Perform 95th percentile normalization. Display the magnitude image using *imshow* grayscale window of [0 max_val] to enable visual comparison. Display the error image with respect to the reference image using *imshow* grayscale window of [0 0.2*max_val]. Compute PSNR and SSIM. Briefly comment on the results.

PART II – SENSE (40 pts)

In Part II, we will use the OLC from **map1** (after 95th percentile normalization) as the *reference image*. Use max_val as the maximum pixel intensity of this image. During PSNR and SSIM computations, normalize both the image of interest and the reference image by max_val.

2.1) Generate Undersampled Images: Write a function that takes the fully sampled images "im", and generates undersampled aliased images and the corresponding k-space data:

[imu, Mu] = undersample(im, Rx, Ry)

This function will take the image *im*, take a centered 2D FFT to compute respective k-space data. The k-space data will be **removed** on decimated lines in the k-space to yield "Mu". Lastly, the function will take an inverse centered 2D FFT to produce the aliased images, "imu". For example, for Rx=1 and Ry=4, the function will **remove** three out of every four **columns**. For Rx=Ry=2 it will **remove** every other row and column. Hence, both "Mu" and "imu" will be of size

(Nx/Rx) × (Ny/Ry) × Nc. Assume that, for convenience, we will only consider the cases where Nx/Rx and Ny/Ry are even numbers.

Test your routine for (Rx,Ry) = (1,2), (1,4), (2,2). For each case, display the magnitude of the images and the corresponding k-space spectrums for all coils. Briefly describe what you see in these undersampled images.

2.2) SENSE Reconstruction: In class, we covered the unregularized version of SENSE, where the resolved source voxels can be expressed as follows (assuming $\Psi = I$):

$$\underline{\hat{m}} = (C^H C)^{-1} C^H \underline{m_s}$$

Here, $\underline{\hat{m}}$ is the vector of resolved source voxels, C is the matrix of coil sensitivities at the source voxels, and $\underline{m_s}$ is the vector of aliased voxel intensities from all coils. A simple improvement of this reconstruction is to incorporate l_2-regularization as follows:

$$\underline{\hat{m}} = (C^H C + \lambda I)^{-1} C^H \underline{m_s}$$

As we have seen previously in linear system of equations, the addition of an identity matrix improves the conditioning of the problem, and hence regularizes the reconstruction.

Write a function that takes the coil sensitivity maps and the aliased, undersampled images (as computed in Part (2.1)), and computes the l_2-regularized SENSE reconstruction of the image:

im_sense = l2sense(imu, map, Rx, Ry, lambda)

Be careful when determining the indices of source voxels when compared to the aliased voxel index. As the last step of reconstruction, take the magnitude of the image, then normalize by its 95th percentile pixel intensity.

Use only **map1** for this question. To test that your function works, first reconstruct for Rx=Ry=1, λ=0. Display the resulting image using a window of [0 max_val], error image using a window of [0 0.2*max_val]. Compute PSNR and SSIM. Verify that this gives exactly the same reconstruction as the OLC method (up to numerical computation error), such that PSNR is extremely large and SSIM is practically equal to 1.

2.3) Use only **map1** for this question. Reconstruct for (Rx,Ry) = (1,2), (1,4), (2,2), for λ = 0. For each case, display the resulting image using a window of [0 max_val], error image using a window of [0 0.2*max_val]. Compute PSNR and SSIM. There should be major problems/artifacts in the reconstructed images without regularization. Briefly comment on the results.

2.4) Use only **map1** for this question. For (Rx,Ry) = (1,2), (1,4), (2,2), select a suitable regularization weight λ for each case (i.e., λ can be different for each case). When selecting λ,

try out λ values in logarithmic scale (i.e., in powers of 10). A suitable λ should suppress the problems/artifacts seen in Part (2.3), without creating new artifacts. State the λ value that you selected for each (Rx,Ry) case and show the results only for the selected λ value.

For each (Rx,Ry), display the resulting image using a window of [0 max_val], error image using a window of [0 0.2*max_val]. Compute PSNR and SSIM. Briefly comment on how λ affects the results. Briefly comment on how the image quality changes with increasing acceleration rates.

2.5) Importance of Accurate Coil Sensitivities: SENSE reconstruction is very sensitive to the accuracy of coil sensitivity maps. Using the l2sense function that you wrote in the previous part, compute SENSE reconstruction using the second set of sensitivity maps, **map2**. Reconstruct the images for (Rx,Ry) = (1,2), (1,4), (2,2). For each case, use the same λ that you used in Part (2.4).

For each case, display the resulting image using a window of [0 max_val], error image using a window of [0 0.2*max_val]. Compute PSNR and SSIM. Briefly compare the results to those from Part (2.4).

2.6) g-Factor for l_2-regularized SENSE: One of the most important characteristics of SENSE reconstruction is the geometry factor g, which tells us how well conditioned the reconstruction problem will be. Using the mathematical formulations that we covered in class, we can go one step further to derive the g-factor for l_2-regularized SENSE.

As given in class, if

$$Cov(\underline{m}_s) = \Psi$$

then

$$Cov(A\underline{m}_s) = A\Psi A^H$$

In this homework, for the unregularized version of SENSE with $\Psi = I$:

$$A = (C^H C)^{-1} C^H$$

which then yields

$$Cov(\underline{\hat{m}}) = AA^H = (C^H C)^{-1}$$

Using this result, we derived the following expression for the g-factor:

$$g = \sqrt{[C^H C]_{i,i}[AA^H]_{i,i}} \quad (1)$$

Here, the first term in the square root is the inverse of the covariance from OLC reconstruction of the fully sampled case, and the second term is the covariance from SENSE reconstruction.

Now, for l_2-regularized SENSE, we have

$$A = (C^H C + \lambda I)^{-1} C^H$$

Derive a closed-form expression for $Cov(\underline{\hat{m}}) = AA^H$ for this case. Then, insert this expression into Eqn. (1) to derive the *g*-factor for l_2-regularized SENSE.

2.7) *g*-Factor Map: Use only **map1** for this question. Using the expression that you derived in Part (2.6), write a function that calculates the *g*-factor:

g = gfactor(map, Rx, Ry,lambda)

Here, g has the same size as the images, i.e., Nx×Ny.

Note: If ALL of the original sensitivity maps have a value zero at a voxel, set *g*-factor to 0 for that voxel. You may find complex-valued *g*-factor due to computational round-off errors before the square-root operation. In any case, the imaginary part should be very small. Hence, use only the real part of the computed *g*-factor.

Compute and display the *g*-factor maps for (Rx,Ry) = (1,2), (1,4), (2,2), with the λ values that you used in Part (2.4). Display the results using *imshow* grayscale window of [0 5] for each case, so that you can visually compare all cases. Briefly comment on the results.

2.8) *g*-Factor Map for Insufficient Regularization: Use only **map1** for this question. With λ values that you used in Part (2.4) multiplied by 10^{-6}, compute and display the *g*-factor maps for (Rx,Ry) = (1,2), (1,4), (2,2). Display the results using *imshow* grayscale window of [0 5] again. Briefly compare these *g*-factor maps with those in Part (2.7).

PART III – GRAPPA (40 pts)

For GRAPPA, we do not need external coil sensitivities. So, we will only use im(Nx,Ny,Nc).

In Part III, we will use the SoS image (after 95th percentile normalization) from Part (1.4) as the *reference image*. Define a variable named max_val as the maximum pixel intensity of this image. During PSNR and SSIM computations, normalize both the image of interest and the reference image by max_val.

3.1) Calibration Images: In practice, it is challenging to accurately measure coil sensitivities since the maps can be affected by various factors including patient configuration. Autocalibrating methods such as GRAPPA have been developed as a solution to this problem. Write a function that takes the fully sampled image "im", and generates calibration images and the corresponding k-space data, i.e.,

[imc, Mc] = imcalib(im, calibx, caliby)

This function will take the image from all coils "im", take a centered 2D FFT to compute respective k-space data. To produce "Mc", the k-space data will be **zeroed out** (i.e., replaced with zeros)

everywhere, with the exception of a calibration region of size calibx × caliby at the center of k-space that will be left intact. Lastly, the function will take centered inverse 2D FFT of "Mc" to produce the calibration images, "imc".

To get a sense of how the calibration region in GRAPPA incorporates coil sensitivity information, test this function for (calibx, caliby) = (8,8), (16,16), (32,32). Display the magnitude of the images for "imc" and the corresponding k-space spectrums for "Mc" from all coils. Briefly describe what you see in the images.

3.2) Undersampled Images with Calibration Region: Write a function that takes the fully sampled images, and generates undersampled aliased images with calibration region and the corresponding k-space data, i.e.,

[imu, Mu] = undersamplecalib(im, Rx, Ry, calibx, caliby)

This function will take the image "im", take a centered 2D FFT to compute respective k-space data. The k-space data will be **zeroed out** (i.e., replaced with zeros) on decimated lines in the frequency domain, with the exception of a calibration region of size calibx × caliby at the center of k-space that will be left intact. These operations will yield "Mu". Lastly, the function will take an inverse 2D FFT to produce the aliased images, "imu". For convenience, assume that the image size is evenly divisible by Rx and Ry.

Test this function for (Rx, Ry) = (1,2), (1,4), (2,2). Use (calibx,caliby) = (32,32) for all cases. Display the magnitude of the images for "imu" and the corresponding k-space spectrums for "Mu" from all coils. Briefly describe what you see in the images.

3.3) GRAPPA Kernel Weights: In this homework, we will only consider the case of (Rx, Ry) = (1,2) for GRAPPA reconstruction. As mentioned in the class, GRAPPA reconstruction gets increasingly complicated for increasing acceleration rates due to sample geometry dependence.

Write a function that takes the central calibration data in k-space to compute GRAPPA kernel weights, i.e.,

kernel =calibrate(Mc,lambda)

Here, Mc is as computed in Part (3.1) (note that you will only use the non-zero parts of Mc), and λ is a regularization parameter used in kernel estimation stage of GRAPPA. Assume a 3×3 neighborhood, such that each unacquired data point has 6 acquired nearest samples from each coil. Then, "kernel" is a 2D matrix of size (6×Nc) × Nc (i.e., 6×Nc GRAPPA coefficients per coil, computed separately for each coil).

Compute the kernel for (calibx,caliby) = (32,32), λ=0. Display the magnitude of the kernel as an image (of size (6×Nc) × Nc). Display the phase of the kernel as an image (of size (6×Nc) × Nc)

3.4) GRAPPA Reconstruction: Write a function that takes undersampled k-space data, and computes the GRAPPA reconstruction of the coil images and the corresponding k-space data for (Rx, Ry) = (1,2), i.e.,

[imr, Mr] = grappa(Mu, Mc, lambda)

Here, Mu and Mc are as computed in Part (3.2) and Part (3.1), respectively. Both "imr" and "Mr" are matrices of size Nx × Ny × Nc.

Perform GRAPPA reconstruction for (calibx,caliby) = (32,32), λ=0. Display the magnitude images and the corresponding k-space data from all coils.

Next, perform a sum-of-squares (SoS) combination of coils, and then normalize the result by its 95th percentile pixel intensity. Display the resulting image using a window of [0 max_val], error image using a window of [0 0.2*max_val]. Compute PSNR and SSSIM. Briefly comment on the results.

3.5) Repeat Part (3.4) by selecting a suitable λ that yields the minimum amount of artifact in reconstructed images (again, try λ values in logarithmic scale when selecting λ). Display the resulting image using a window of [0 max_val], error image using a window of [0 0.2*max_val]. Compute PSNR and SSSIM. Briefly comment on the quality of these results when compared to λ=0 case. Did regularization provide a significant improvement in this case?

3.6) Final Remarks: Briefly comment on the quality of the images obtained from SENSE to those obtained via GRAPPA at matching acceleration factors. Briefly discuss the relative benefits and disadvantages of the two techniques.